CSC-239 *Java Programming*

Chapter 10: Object-Oriented Thinking

October 8, 2025

NOTICE to Students: Portions of these lecture slides are
Copyright 2021 Pearson Education, Inc.
We have a license to use this material ***only*** in the context of this course.
There is NO PERMISSION to share these lecture slides with anyone not taking the course.
There is NO PERMISSION to post these lecture slides on the Internet.

Reminder: Quiz 2

- Quiz 2 began on Monday, October 6.
  - Online (Moodle) Quiz.
  - The Quiz closes on Monday Oct. 13 at 11:00 p.m..
  - Each student will have two hours to complete the Quiz, from the time that they begin.
- The material covered on Quiz 2:
  - The lectures of September 10 through 29.
  - Chapters 4 through 7.
  - The quiz will include True / False, Multiple-Choice questions.

Reminder: Project 1 due October 15

- Project 1 is due in one week: Wednesday, October 15.
- As stated in the *Syllabus*, the grade penalty for turning in a project late is:

  1% per day (after a 1-day grace period), up to a maximum grade penalty of 20%.

  Any project that is 21 days late (or more) receives a grade of ZERO.
- What this means is: if you need a few extra days to do a good job on your project, then the late penalty will not be very much.
- However, if you need significantly more time than that, you should meet with the instructor to get some help.

Special NOTE Regarding Quiz 2 (1)

Several questions in **Quiz 2** require that you study the source code of a small Java program, and answer the question: "What is the output of this program?"

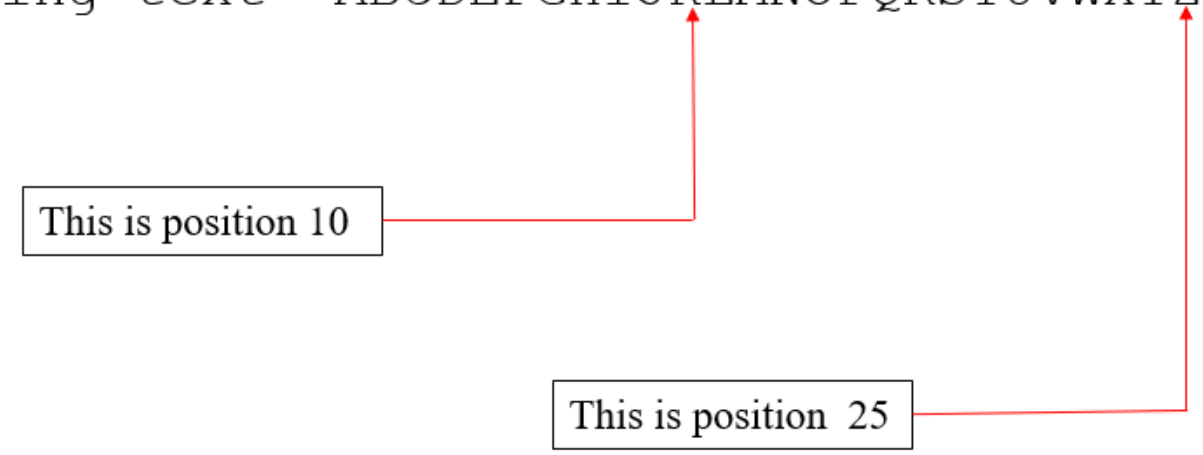
- To answer the question correctly, you need to read the program carefully and *manually* keep track of the program's "current" position in a **String** that is part of the code.
- To make this task easier, the program includes a special comment that helps you to see the exact position of each character in the **String**.
- The next slide contains an example that shows what this special comment looks like:

Special NOTE Regarding Quiz 2 (2)

The '**K**' character is at position 10 in the **String**.

The '**Z**' character is at position 25 in the **String**.

```
// These numbers are intended as a help
// while you are keeping track of the
// position within a String object:
//                        1         2          3
//              0....5....0....5....0....5....0
String text="ABCDEFGHIJKLMNOPQRSTUVWXYZ";
```



Online Information

- Java Tutorials:

  https://docs.oracle.com/javase/tutorial/

- Java API:

  https://docs.oracle.com/en/java/javase/21/docs/api/index.html

- The official "*Grader Than*" documentation:

  https://docs.graderthan.com/student/

- *Grader Than* Support E-mail:

  support@graderthan.com

Class Abstraction and Encapsulation

- Class abstraction: to separate the class implementation from the use of the class.
- Description of the class: lets the user know how the class can be used.
- The user of the class does ***not*** need to know how the class is implemented.
- The details of the implementation:
  - encapsulated
  - hidden from the user.

The Class's Public Interface

- The features of a class which can be "seen" by a program that uses the class. The "Class Contract" (between the class creator and its users):
  - Signatures of public methods
  - Public constants.
- The client programs use the class through the class contract.
- The internal details of the class implementation are ***not*** generally part of its public interface
  - Implementation details are like a "black box", hidden from the clients.

Object-Oriented Thinking

- Classes provide flexibility and modularity for building reusable software.
- Data and Behavior are tightly coupled.
  - Data and Operations are logically linked (in the class).

Class Relationships

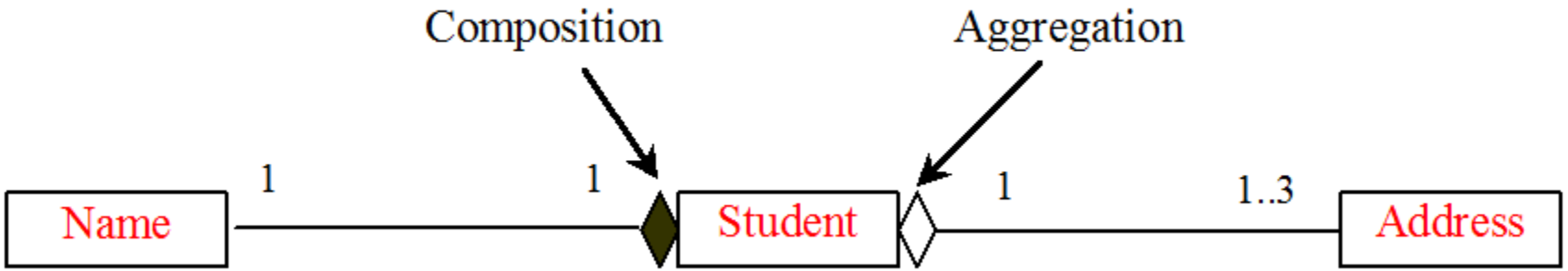
Section 10.4 of the textbook (pages 375-377) discusses some of the ways an object of one class may interact with objects of another class.

- **Association**: is a general binary relationship that describes an activity between two classes:
  - A Student may take a Course.
  - A Faculty member may teach a course.
- This can be illustrated in a UML diagram as shown below:



Object Aggregation and Composition

- **Aggregation** models *has-a* relationship
  - Represents ownership relationship between two objects.
  - The owner object is called an *aggregating object* and its class an *aggregating class*.
  - The subject object is called an *aggregated object* and its class an *aggregated class*.
- **Composition**: special case of *aggregation*.
  - Composition implies exclusive ownership.



Aggregation: Class Representation

Aggregation relationship:

- Usually represented as a data field in the aggregating class.
- For example, the relationship in the previous slide can be represented as follows:

```
public class Name {
  ...
}
```

Aggregated class

```
public class Student {
  private Name name;
  private Address address;

  ...
}
```

Aggregating class

```
public class Address {
  ...
}
```

Aggregated class

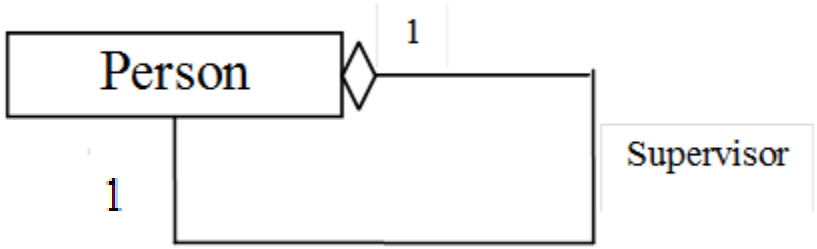
Aggregation or Composition

- Composition is often a "whole-part" relationship:
  - A car **HAS-A** number of doors (usually 2 or 4).
  - A car **HAS-A**n engine.
- Composition implies exclusive ownership:
  - A person **HAS A** brain.

- The distinction between **Aggregation** and **Composition** is not represented in the syntax of the way the class is defined, but rather in the way objects of the *aggregating* class interact with objects of the *aggregated* class.

Aggregation Between Same Class (1)

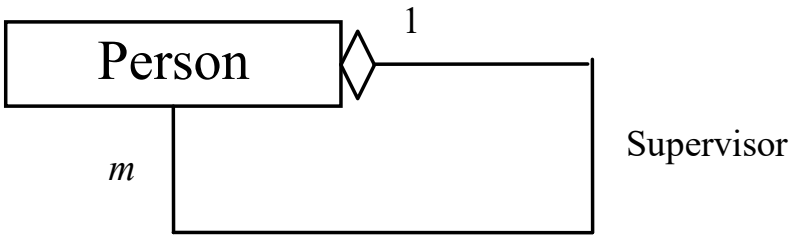
- Aggregation may exist between objects of the same class: for example, a person may have a supervisor.




```
public class Person {
  // The type for the data is the class itself
  private Person supervisor;
  ...
}
```

Aggregation Between Same Class (2)

What happens if a person has several supervisors?




```
public class Person {
   ...
   private Person[] supervisors;
}
```

Wrapper Classes

- Boolean
- Character
- Short
- Byte
- Integer
- Long
- Float
- Double

NOTE:

(1) The wrapper classes do not have no-arg constructors.

(2) The instances of all wrapper classes are immutable, i.e., their internal values cannot be changed once the objects are created.

Numeric Wrapper Class Constructors

You can construct a wrapper object either from a primitive data type value or from a string representing the numeric value. The constructors for Integer and Double are:

public Integer(int value)

public Integer(String s)

public Double(double value)

public Double(String s)

Numeric Wrapper Class Constants

- `MAX_VALUE`
  - represents the maximum value of the corresponding primitive data type.
- `MIN_VALUE`
  - represents the minimum byte, short, int, and long values.
  - represents the minimum *positive* float and double values.

Numeric Conversion Methods

Each numeric wrapper class implements methods that "convert" objects into primitive type values:

```
doubleValue()
floatValue()
intValue()
longValue()
shortValue()
```

The Static valueOf Methods

`valueOf(String s)`

creates a new object initialized to the value represented by the specified string. For example:

```
Double number = Double.valueOf("12.4");
```

```
Integer intval =  Integer.valueOf("12");
```

Parsing Strings into Numbers

Methods to convert strings to primitive types:

```
static int Integer.parseInt(String s)
static int Integer.parseInt(String s, radix)

static double Double.parseDouble(String s)
static double Double.parseDouble(String s,
                                 radix)
```

Boxing, Unboxing

- Converting a primitive value to a wrapper object is called *boxing*.
- Converted a wrapper object to a primitive value is called *unboxing*.
- In some contexts, this conversion is done automatically, for example:

```
Integer intObject = 2;
```

- This is called *autoboxing*, or *autounboxing*.

Using the **`printf()`** method

`printf` (*format_string*`,` *var1*`,` *var2*`,` ...)

where *format_string* is a quoted string containing plain text and a "format specifier" field for each variable to print.

For example:

```
System.out.printf("var1=%f, var2=%f)\n",
                  var1, var2);
```

Finer details, such as field width, are coded between the **`%`** and the **`f`** as shown:

`%8.3f`

The `String` Class

- Constructing a String:

```
String message = "Welcome to Java";

String message =
         new String("Welcome to Java");

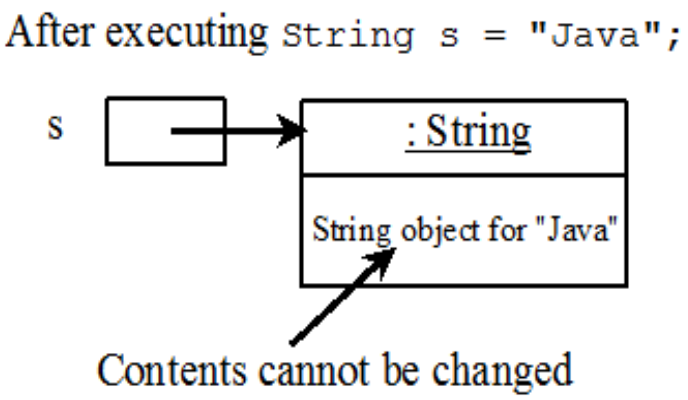
String s = new String();
```

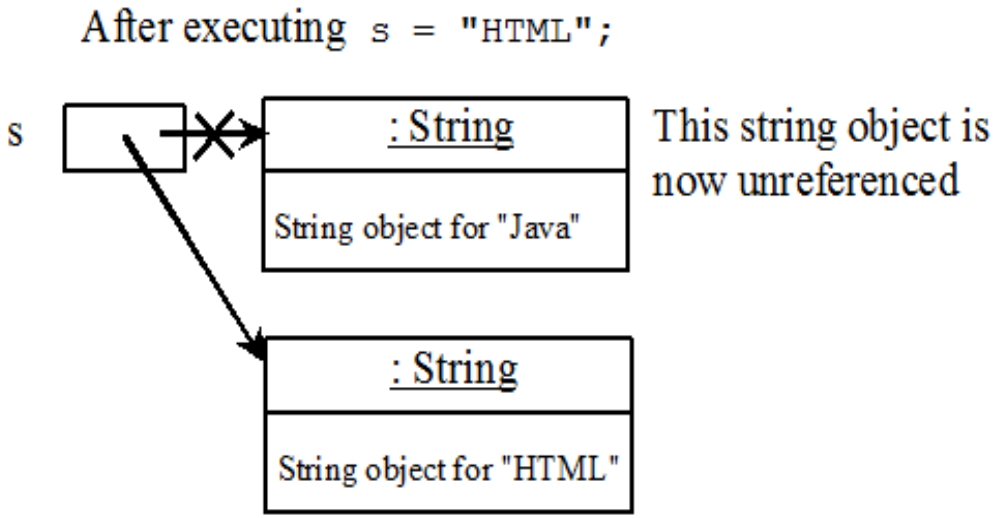
- A String is an object:
  - It is ***not*** simply an array of characters.
  - Although there is probably an array of characters somewhere inside the implementation of a String, we cannot access it directly.
  - We can only get at the text data inside a String by using its methods.

Strings Are Immutable

- A String object is immutable: once it has been created, its contents cannot be changed.
- Really? What about the following example?

```
String s = "Java";
s = "HTML";
```





Interned Strings

- String literals (for example: **"Hello"**) are reused to save memory.
- Called an "interned" string.
- If you use the string initializer, no new object is created if the interned object has already been created.

```
String greeting = "Hello";
```

- A new object is created if you use the **new** operator.

Interned Strings -- Examples

```
String s1 = "Welcome to Java";
String s2 = new String("Welcome to Java");
String s3 = "Welcome to Java";
System.out.println("s1==s2 is " + (s1==s2));
System.out.println("s1==s3 is " + (s1==s3));
```

Output of this code is:

```
s1==s2 is false
s1==s3 is true
```

String Comparisons

- The **`equals`** method:

```
String s1 = new String("Welcome“);
String s2 = “Welcome";

if (s1.equals(s2)){
  // s1, s2 have the same contents
}

if (s1 == s2) {
  // s1 and s2 have the same reference
}
```

More String Comparisons

- compareTo(Object object)

```
String s1 = new String("Welcome“);
String s2 = “Welcome";

if (s1.compareTo(s2) > 0) {
  // s1 is greater than s2
}
else if (s1.compareTo(s2) == 0) {
  // s1, s2 have the same contents
}
else
   // s1 is less than s2
```

Finding String Length

Finding string length using the `length()` method:

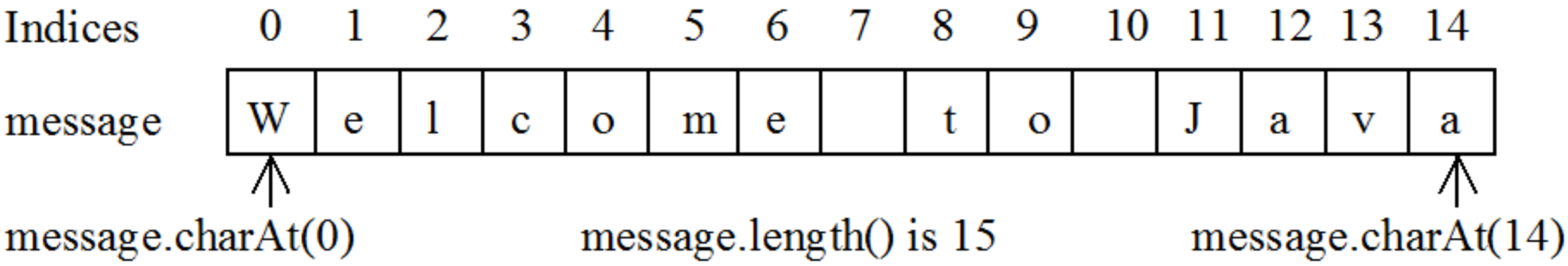
```
message = "Welcome";
message.length() (returns 7)
```

Do <u>not</u> confuse this with the length of an array:

```
arrayName.length
```

Individual Characters in a String

- Do <u>not</u> use `message[index]`
- Use `message.charAt(index)`
- Index starts from `0`



String Concatenation

```
String s3 = s1.concat(s2);
```

same as

```
String s3 = s1 + s2;
```

This neat and concise expression:

```
s1 + s2 + s3 + s4 + s5
```

is equivalent to:

```
(((s1.concat(s2)).concat(s3))
 .concat(s4)).concat(s5);
```

String Concatination Example

```
String s1 = "air";
String s2 = "line";
String s3 = s1 + s2;
System.out.println("s3 = " + s3);
```

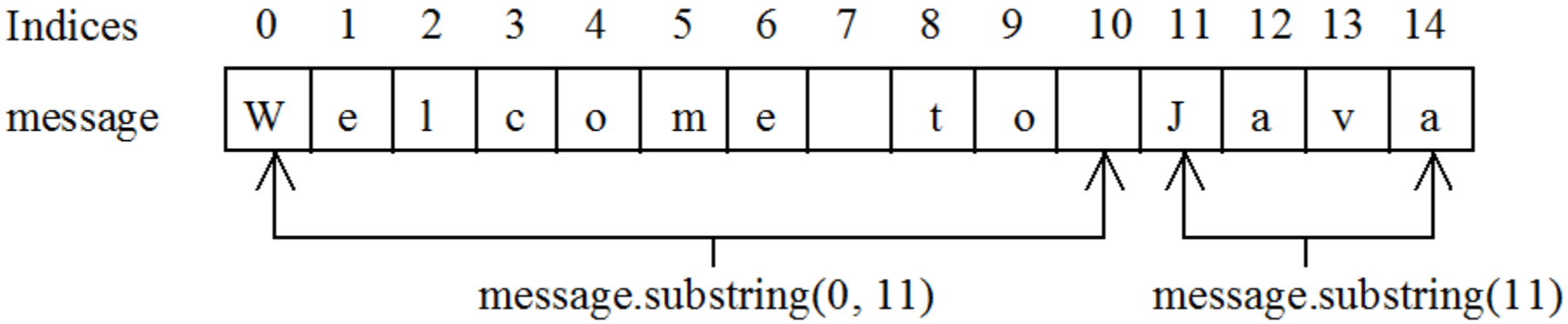
Output of this code is:

```
s3 = airline
```

Extracting Substrings

You can extract a single character from a string using the charAt method. You can also extract a substring from a string using the substring method in the String class.

```
String s1 = "Welcome to Java";
String s2 = s1.substring(0, 11) + "HTML";
```



Converting and Replacing Strings

```
"Welcome".toLowerCase()
```
returns `welcome`

```
"Welcome".toUpperCase()
```
returns `WELCOME`

```
"  Welcome  ".trim()
```
returns `Welcome`

```
"Welcome".replace('e', 'A')
```
returns `WAlcomA`

```
"Welcome".replaceFirst("e", "AB")
```
returns `WABlcome`

```
"Welcome".replace("e", "AB")
```
returns `WABlcomAB`

```
"Welcome".replace("el", "AB")
```
returns `WABcome`

Important: Splitting a String

```
String[] tokens =
  "Java#HTML#Perl".split("#", 0);
for (int i = 0; i < tokens.length; i++){
  System.out.print(tokens[i] + " ");
}
```

This code displays

```
Java HTML Perl
```

The second parameter for the **split** method is the maximum number of times the pattern will be applied (0=unlimited).

See also: API documentation for the **`String`** class.

Regular Expressions

You can match, replace, or split a string by specifying a pattern.

```
"Java".matches("Java");
"Java".equals("Java");


"Java is fun".matches("Java.*");
"Java is cool".matches("Java.*");
```

See also: **Appendix H** in our textbook.

More Regular Expressions

The following statement returns a new string that replaces $, +, or # in

```
"a+b$#c"
```

by the string NNN.

```
String s =
    "a+b$#c".replaceAll("[$+#]", "NNN");
System.out.println(s);
```

The regular expression "`[$+#]`" specifies a pattern that matches $, +, or #. So, the output is

```
aNNNbNNNNNNc
```

Still More Regular Expressions

The following statement splits the string into an array of strings delimited by some punctuation marks.

```
String[] tokens =
"Java,C?C#,C++".split("[.,:;?]");

for (int i = 0; i < tokens.length; i++)
  System.out.println(tokens[i]);
```

Displays:

```
Java
C
C#
C++
```

The Index of Character or Substring

```
"Welcome to Java".indexOf('W')
```
returns 0.

```
"Welcome to Java".indexOf('x')
```
returns -1 .

```
"Welcome to Java".indexOf('o', 5)
```
returns 9.

```
"Welcome to Java".indexOf("come")
```
returns 3.

```
"Welcome to Java".indexOf("Java", 5)
```
returns 11.

```
"Welcome to Java".indexOf("java", 5)
```
returns -1.

```
"Welcome to Java".lastIndexOf('a')
```
returns 14.

Converting Data to Strings

valueOf( ) methods:

Convert data of various types to String:

For example:

```
String.valueOf(5.44)
```

Returns a string containing the characters

‘**5**’, ‘**.**’, ‘**4**’, and ‘**4**’.

The `Character` Class Examples

```
Character charObject = new Character('b');
```

```
charObject.compareTo(new Character('a'))
```
returns 1

```
charObject.compareTo(new Character('b'))
```
returns 0

```
charObject.compareTo(new Character('c'))
```
returns -1

```
charObject.compareTo(new Character('d')
```
returns –2

```
charObject.equals(new Character('b'))
```
returns true

```
charObject.equals(new Character('d'))
```
returns false

StringBuilder, StringBuffer

- Alternatives to String
- Unlike Strings, they are *not* “immutable”.
- You can add, insert or append to a StringBuilder or StringBuffer object.
- StringBuilder is more efficient than StringBuffer
- StringBuffer is “Synchronized”, meaning it is OK for multiple processes to use the same code at the same time.